Using konghq.com as an AI governance service and need to set up a service to categorizes risk levels with AI. Can you point me in right directions using Kong?

I'm pulling together the Kong-native pieces that fit AI risk categorization and enforcement, then I'll map them into a practical service design you can implement.

I found the main Kong pieces you'd use: AI Gateway for the front door, Prompt Guard and Semantic Prompt Guard for traffic screening, AI PII Sanitizer for sensitive data handling, and OPA if you want policy-based risk decisions outside of app code. Now I'm turning that into a service pattern for "risk classify then enforce."

Thought for 37s

Yes. The clean Kong pattern is: **use Kong to enforce AI governance, but keep "risk classification" as a separate decision step** that feeds Kong policies.

A good starting architecture is:

client/app -> Kong AI Gateway -> risk classifier/policy decision -> Kong enforcement plugins -> LLM provider(s)

Why this fits Kong:
Kong AI Gateway gives you a standard front door for LLM traffic through **AI Proxy** and **AI Proxy Advanced**. AI Proxy standardizes a few OpenAI-style request formats and proxies them to supported providers, while AI Proxy Advanced can route to **multiple models/providers at once** for load balancing and policy-driven routing.

The simplest way to do risk levels with Kong

Think in **two layers**:

1. Classify risk
Have a small internal service or policy engine decide a risk level like:

- low
- medium
- high
- blocked

That decision can use:

- tenant / app / user role
- model requested
- prompt text
- presence of sensitive data
- route or business workflow
- tool usage / MCP usage
- geography / compliance flags

2. Enforce in Kong
Once risk is known, Kong applies different controls per level.

That usually means:

- **low**: allow, cheaper/faster model
- **medium**: allow + stronger prompt checks + sanitize + lower token/rate budgets
- **high**: allow only on approved models, log heavily, maybe redact input/output
- **blocked**: reject before provider call

Kong features that map well to this

AI Proxy / AI Proxy Advanced
Use these as the main AI ingress. AI Proxy handles provider abstraction; AI Proxy Advanced adds multi-target routing and load balancing, which is useful if you want "high-risk goes only to private Bedrock/Azure/OpenAI-approved targets."

AI Prompt Guard
Use this for explicit regex-style controls such as:

- banned terms
- secret patterns
- jailbreak phrases
- internal project names
- "ignore previous instructions" style attacks

It scans llm/v1/chat and llm/v1/completions traffic and requires AI Proxy or AI Proxy Advanced in front of it.

AI Semantic Prompt Guard
Use this when risk is about **meaning**, not exact wording. It can allow or deny prompts based on semantic similarity to known examples, and deny rules take precedence over allow rules. It also depends on AI Gateway and uses vector similarity.

AI PII Sanitizer
Use this to redact or replace sensitive data in requests and responses. It supports request sanitization, response sanitization, or both, and can restore original values in responses when configured. It requires AI Proxy or AI Proxy Advanced.

AI AWS Guardrails / provider safety plugins
If you already use Bedrock guardrails or similar provider-native controls, Kong can enforce those in the path too. Kong's AWS Guardrails how-to specifically shows AI Proxy Advanced in front of model routing, then the guardrail plugin blocking unsafe inputs and outputs.

AI Rate Limiting Advanced
Useful for "risk-budgeting." It supports policy-based AI rate limiting and can match on headers, which is handy if your classifier stamps something like X-AI-Risk-Level: high.

Observability / audit
Kong AI Gateway exposes audit logs, LLM metrics, token usage, latency, and cost telemetry, and supports OpenTelemetry-based tracing for AI traffic. That is important if your governance program needs evidence and reviews.

How to implement the classifier decision

You have three realistic options in Kong:

Best enterprise pattern: external policy engine
Use **OPA** or a small internal "AI risk scoring" service. Kong's own guidance for OPA is about centralizing policy logic for security, compliance, consistency, and agility at the gateway.

In practice:

- Kong receives the request
- your policy service calculates risk
- it returns a decision like:
    - risk_level=high
    - allowed_models=["bedrock-claude","azure-gpt4o"]
    - sanitize=true
    - max_tokens=1000
    - requires_audit=true

Then Kong enforces based on that.

Fastest prototype: Pre-Function plugin
Kong's **Pre-Function** plugin can run Lua before other plugins. Kong documents using it to inspect a request and either let it through or terminate it. This is useful for a prototype that tags requests or blocks obvious cases.

Most maintainable long-term: custom plugin
Kong supports building custom plugins. That is the right move if AI risk classification becomes a core control and you want a first-class plugin that calls your classifier, stamps headers, rewrites route targets, and logs decisions consistently.

A practical control model

I'd set it up like this:

Route 1: /ai/chat
This is your single managed entrypoint.

Step A: identify caller
Use your normal Kong auth stack so every request is tied to a consumer, app, tenant, or service account.

Step B: classify
Call your risk engine or OPA and produce:

- X-AI-Risk-Level
- X-AI-Data-Class
- X-AI-Approved-Model-Group
- X-AI-Audit-Mode

Step C: enforce by policy
Example:

For low

- AI Proxy Advanced routes to standard model pool
- normal rate limits
- log metrics

For medium

- AI Prompt Guard
- AI Semantic Prompt Guard
- AI PII Sanitizer on request
- tighter AI Rate Limiting Advanced policy

For high

- only approved provider targets
- provider guardrails
- request + response sanitization
- stricter token and cost ceilings
- stronger audit trail

For blocked

- terminate at gateway with a policy message

What Kong does well here

Kong is very strong at:

- central entrypoint
- provider abstraction
- policy enforcement
- prompt/output safety controls
- data sanitization
- rate/cost controls

- logging and auditability

What Kong does not give you out of the box

Kong does **not** appear to provide a built-in "AI risk categorization service" that decides your custom business risk tiers for you. The better model is:

- **your classifier decides**
- **Kong enforces** the result

That separation is usually better anyway, because risk logic changes faster than gateway plumbing.

Recommended first build

I'd start with this minimal stack:

1. **AI Proxy Advanced** as the universal AI entrypoint.
2. A small **risk-classifier API** or **OPA policy** that returns a risk tier.
3. Add a header like X-AI-Risk-Level.
4. Attach:
    - **AI Prompt Guard**
    - **AI Semantic Prompt Guard**
    - **AI PII Sanitizer**
    - **AI Rate Limiting Advanced**
5. Send audit/usage data to Konnect or your SIEM/Otel stack.

Good docs to open first

Start with these official Kong docs:

- **AI Gateway overview** and **AI Proxy / AI Proxy Advanced** for the base architecture.
- **AI Data Governance** for safety, observability, and governance plugins.
- **AI Prompt Guard**, **AI Semantic Prompt Guard**, and **AI PII Sanitizer** for enforcement.
- **OPA with Kong** if you want policy-driven risk decisions.

If you want, I can sketch a **reference architecture and sample Kong config** for a 3-tier policy like low / medium / high using AI Proxy Advanced + OPA-style decisions.

sketch a reference architecture and sample Kong config

I'm turning this into something you can actually build: a reference flow, then a concrete Kong config pattern with risk tiers, routing, and enforcement points.

Thought for 1m 15s

Here's a solid **reference architecture** and a **sample decK config** you can adapt.

The key design choice is this: **Kong should be the enforcement plane, while your risk engine decides the tier**. Kong AI Gateway gives you the AI ingress point through **AI Proxy / AI Proxy Advanced**, and Kong's AI governance plugins then enforce regex checks, semantic topic checks, PII sanitization, and token/cost-based throttling. The OPA plugin is the clean way to externalize policy decisions if you want a policy engine instead of hard-coding logic.

Reference architecture

```
        ┌──────────────────────────────┐
        │  App / Portal / Workflow     │
        │  sends OpenAI-style request  │
        └──────────────┬───────────────┘
                       │
                       ▼
        ┌──────────────────────────────┐
        │ Kong Route: /ai/chat         │
        │ AuthN/AuthZ + consumer ID    │
        └──────────────┬───────────────┘
                       │
                       ▼
        ┌──────────────────────────────┐
        │ Risk decision                │
        │ Option A: app sets header    │
        │   X-AI-Risk-Level            │
        │ Option B: Kong calls OPA     │
        │ Option C: custom plugin      │
        └──────────────┬───────────────┘
                       │
         ┌─────────────┼─────────────┐
         │             │             │
         ▼             ▼             ▼
  low-risk path   med-risk path  high-risk path
  cheaper model   more checks    approved models only
  light limits    sanitize req   sanitize req+resp
                  regex+semantic strict limits + audit
         │             │             │
         └─────────────┼─────────────┘
                       │
                       ▼
        ┌──────────────────────────────┐
        │ AI Proxy Advanced            │
        │ Route/load-balance to        │
        │ OpenAI / Azure / Bedrock /   │
        │ Anthropic / etc.             │
        └──────────────┬───────────────┘
                       │
                       ▼
        ┌──────────────────────────────┐
        │ Logs / metrics / audit       │
        │ OTEL / Konnect / SIEM        │
        └──────────────────────────────┘
```

Risk contract

Have your classifier return something like:

```
{
  "risk_level": "high",
  "allow": true,
  "approved_model_group": "restricted",
  "sanitize_input": true,
  "sanitize_output": true,
  "max_tokens": 1000,
  "audit_mode": "full"
}
```

Then stamp at least:

- X-AI-Risk-Level: low|medium|high|blocked
- X-AI-Model-Group: standard|restricted
- X-AI-Audit-Mode: basic|full

Best deployment pattern

For production, I'd use one of these:

Best: app or internal policy service computes the risk tier first, then calls Kong with the risk header.
Also good: Kong OPA plugin authorizes or blocks based on policy.
Prototype only: Kong Pre-Function can run Lua before other plugins, but Kong explicitly warns that Pre-Function allows arbitrary code execution for anyone who can enable it.

Sample Kong config

Below is a **practical skeleton** in decK format. The parts for ai-proxy-advanced, ai-prompt-guard, ai-sanitizer, and ai-rate-limiting-advanced follow Kong's documented patterns. I'm using **header-based risk enforcement**, because AI Rate Limiting Advanced explicitly supports policies matched on headers. AI Proxy Advanced uses config.targets and supports multiple providers/models behind one entry point.

```
_format_version: "3.0"

services:
  - name: ai-governance-entry
    url: http://httpbin.konghq.com/anything

routes:
  - name: ai-chat
    service: ai-governance-entry
    paths:
      - /ai/chat
    protocols:
      - http
      - https
    methods:
      - POST
```

```
plugins:
  #
  # 1) Main AI ingress and model routing
  #
  - name: ai-proxy-advanced
    route: ai-chat
    config:
      targets:
        # Standard pool
        - route_type: llm/v1/chat
          auth:
            header_name: Authorization
            header_value: Bearer ${{ env "DECK_OPENAI_API_KEY" }}
          model:
            provider: openai
            name: gpt-4o-mini
            options:
              max_tokens: 800
              temperature: 0.3

        # Restricted / premium pool
        - route_type: llm/v1/chat
          auth:
            header_name: api-key
            header_value: ${{ env "DECK_AZURE_OPENAI_API_KEY" }}
          model:
            provider: azure
            name: gpt-4o
            options:
              max_tokens: 1200
              temperature: 0.2

  #
  # 2) Medium/high prompt regex controls
  # Attach this only if you want it to run for all requests on this route.
  # If you want different controls by risk level, put low/medium/high on
  # separate routes or separate consumer groups and bind plugins there.
  #
  - name: ai-prompt-guard
    route: ai-chat
    config:
      allow_patterns:
        - "(?i).*summari[sz]e.*"
        - "(?i).*classif(y|ication).*"
        - "(?i).*extract.*"
        - "(?i).*review.*"
        - "(?i).*analy[sz]e.*"
```

```
    deny_patterns:
      - "(?i).*bypass.*(policy|guardrail|auth).*"
      - "(?i).*ignore previous instructions.*"
      - "(?i).*disable safety.*"
      - "(?i).*exfiltrat(e|ion).*"
      - "(?i).*credit card.*"
      - "(?i).*social security.*"

#
# 3) PII sanitization
# Typical for medium/high risk. If you need different behavior per tier,
# split the tiers across routes or use a custom plugin / policy service.
#
- name: ai-sanitizer
  route: ai-chat
  config:
    anonymize:
      - phone
      - general
    host: host.docker.internal
    port: 8080
    redact_type: synthetic
    stop_on_error: true
    recover_redacted: false

#
# 4) Risk-tier budget controls using the risk header
# Kong documents header matching in AI Rate Limiting Advanced.
#
- name: ai-rate-limiting-advanced
  route: ai-chat
  config:
    policies:
      - match:
          - type: header
            key: X-AI-Risk-Level
            values:
              - low
        limits:
          - limit: 300
            window_size: 60
          - limit: 10000
            window_size: 3600

      - match:
          - type: header
            key: X-AI-Risk-Level
            values:
```

```
          - medium
      limits:
        - limit: 120
          window_size: 60
        - limit: 3000
          window_size: 3600

    - match:
        - type: header
          key: X-AI-Risk-Level
          values:
            - high
      limits:
        - limit: 30
          window_size: 60
        - limit: 600
          window_size: 3600
```

How to use that config

Option 1: two-step flow

Your app does:

1. POST /risk/classify
2. gets risk_level=medium
3. POST /ai/chat with X-AI-Risk-Level: medium

This is the simplest and most reliable setup.

Option 2: OPA as decision point

Use Kong's OPA plugin to call OPA and return allow/deny decisions. Kong's OPA docs show the core pattern: run OPA server, write policy, then configure config.opa_host and config.opa_path.

That looks like:

```
plugins:
  - name: opa
    route: ai-chat
    config:
      opa_host: opa.internal
      opa_port: 8181
      opa_protocol: http
      opa_path: /v1/data/ai/risk/allow
```

And your Rego policy can decide things like:

- block requests from certain consumers
- block high-risk data to non-approved model groups
- require only specific consumers to use X-AI-Risk-Level: high

Example Rego sketch:

```
package ai.risk

default allow = false

approved_high_risk_consumers := {"governance-app", "fraud-review"}

risk := input.request.http.headers["x-ai-risk-level"]
consumer := input.request.http.headers["x-consumer-id"]

allow if {
  risk == "low"
}

allow if {
  risk == "medium"
}

allow if {
  risk == "high"
  consumer == approved_high_risk_consumers[_]
}
```

That gives you **policy-based authorization**, but it does not automatically rewrite plugin behavior by tier. For dynamic per-tier plugin behavior in one hop, you usually end up with **multiple routes/consumer-groups** or a **custom plugin**.

The configuration pattern I recommend

Pattern A: easiest to operate

Use **three Kong routes** behind separate internal entry points:

- /ai/chat/low
- /ai/chat/medium
- /ai/chat/high

Your classifier rewrites to one of those, and each route has its own plugin stack:

- low: AI Proxy Advanced + light rate limits
- medium: + Prompt Guard + Sanitizer
- high: + stricter Prompt Guard + Sanitizer + semantic controls + reduced model pool

This is easier than trying to make one route behave differently for every plugin.

Pattern B: best UX for clients

Expose one public endpoint /ai/chat, but inside Kong use:

- custom plugin or pre-function to call your risk engine

- route internally to low/medium/high services
- stamp audit headers
- enforce appropriate plugin stack

That gives the cleanest client experience, but it is more engineering.

Good first rollout

Start with:

1. **Single /ai/chat route**
2. **AI Proxy Advanced** with two targets: standard and restricted model pools
3. app-generated X-AI-Risk-Level
4. **AI Rate Limiting Advanced** matched on that header
5. **AI Prompt Guard** for known bad patterns
6. **AI PII Sanitizer** for request redaction
7. later add **AI Semantic Prompt Guard** for topic-level enforcement, since deny rules take precedence over allow rules in the documented examples

Important limitation

One thing to be aware of: **Kong has the enforcement pieces, but not a built-in "business risk classifier" for your custom low/medium/high taxonomy**. AI Prompt Guard, Semantic Prompt Guard, and PII Sanitizer are governance controls, not a full risk scoring engine. AI Proxy Advanced gives you multi-target routing, but the actual risk decision still needs to come from your app, OPA, or a custom plugin.